CENG322 - PA1 Report

Ahmet Kurt
290201034

March 21, 2025

1. Introduction

This report explains the steps that I take to execute PolyBench-C benchmarks on the UHEM system. The purpose of this assignment is to:

- Compile and run five PolyBench-C benchmarks.
- Submit and execute the programs on UHEM.
- Collect execution times over 10 runs.
- Process and compute average execution times.

2. Selected Benchmarks

The following benchmarks were selected:

- **2mm** (Linear Algebra - Kernels)
- **3mm** (Linear Algebra - Kernels)
- **atax** (Linear Algebra - Kernels)
- **bicg** (Linear Algebra - Kernels)
- **doitgen** (Linear Algebra - Kernels)

3. Connecting the VPN

Firstly I downlaoded the file named `openvpn-uhem-config-linux.ovpn` with my username and password. I move to the directory and run it with command `sudo openvpn`.

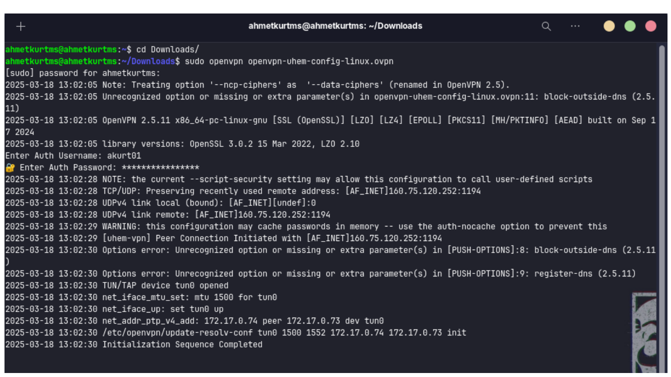


Figure 1: Connecting the VPN

4. Connecting to the UHEM System

After the VPN connection with command: `ssh username@10.128.2.32` and with my password, connected to the system.

Figure 2: Connecting the UHEM

5. Downloading the Polybench C Benchmark

With the command `wget` and `tar`, I downloaded the link which given in the homework PDF file. And extract it in the system.

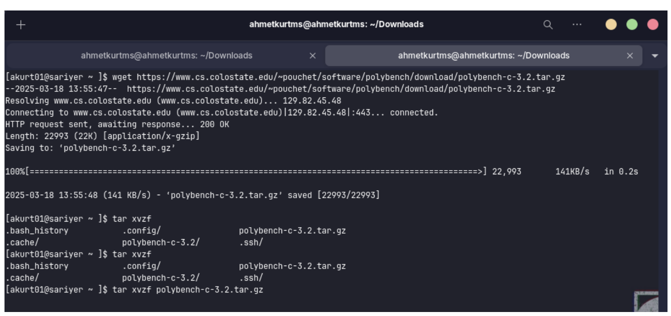


Figure 3: Download-Extract Benchmark

6. Compilation of Benchmarks

After navigating to the directory, I compiled the benchmarks using the `gcc` command with optimization level -O3. Here are the commands I used:

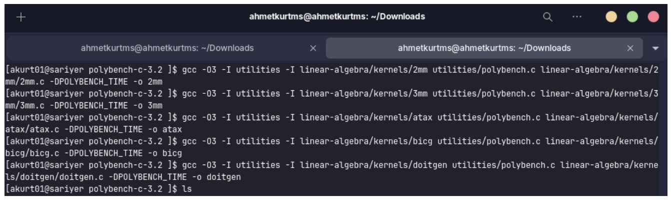


Figure 4: Compiling Benchmarks

7. compute.sh

To automate execution, I used the `compute.sh` script. It runs each benchmark 10 times, measures execution time, and stores results in `output_shell.txt`.

Figure 5: compute.sh

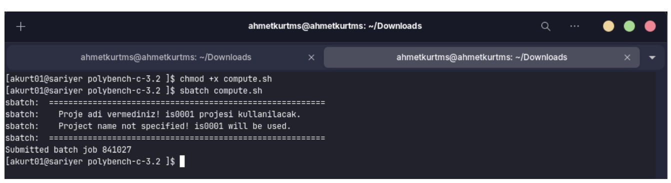


Figure 6: Run the script

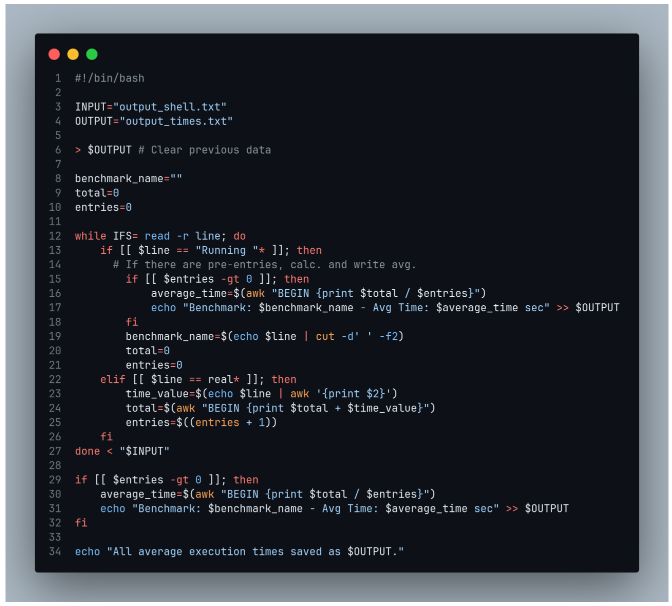
After that, it select the project name automatically. I waited for the run and the slurm - output files exist.

8. Output_shell.txt

```
Running 2mm
real 5.47
real 5.47
real 5.48
real 5.47
real 5.47
real 5.47
real 5.48
real 5.47
real 5.47
real 5.44
Running 3mm
real 8.21
real 8.19
real 8.20
real 8.19
real 8.19
real 8.19
real 8.19
real 8.19
real 8.20
real 8.19
Running atax
real 0.10
real 0.08
real 0.08
real 0.08
real 0.08
real 0.08
real 0.08
real 0.08
real 0.08
real 0.08
Running bicg
real 0.12
real 0.10
real 0.10
real 0.10
real 0.10
real 0.10
real 0.10
real 0.10
real 0.10
real 0.10
Running doitgen
real 0.37
real 0.36
real 0.36
real 0.36
real 0.36
real 0.36
real 0.36
real 0.36
real 0.36
real 0.36
```

9. process.sh

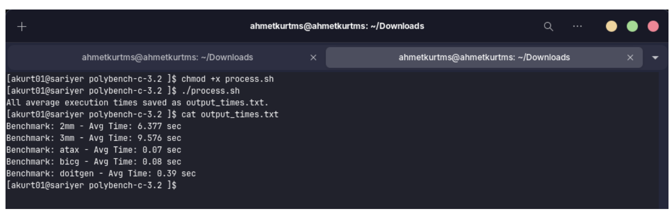
After execution, ‘process.sh‘ extracts execution times from ‘outputshell.txt‘ and computes the average for each benchmark.



```bash
#!/bin/bash

INPUT="output_shell.txt"
OUTPUT="output_times.txt"

> $OUTPUT # Clear previous data

benchmark_name=""
total=0
entries=0

while IFS= read -r line; do
    if [[ $line == "Running "* ]]; then
      # If there are pre-entries, calc. and write avg.
        if [[ $entries -gt 0 ]]; then
            average_time=$(awk "BEGIN {print $total / $entries}")
            echo "Benchmark: $benchmark_name - Avg Time: $average_time sec" >> $OUTPUT
        fi
        benchmark_name=$(echo $line | cut -d' ' -f2)
        total=0
        entries=0
    elif [[ $line == real* ]]; then
        time_value=$(echo $line | awk '{print $2}')
        total=$(awk "BEGIN {print $total + $time_value}")
        entries=$((entries + 1))
    fi
done < "$INPUT"

if [[ $entries -gt 0 ]]; then
    average_time=$(awk "BEGIN {print $total / $entries}")
    echo "Benchmark: $benchmark_name - Avg Time: $average_time sec" >> $OUTPUT
fi

echo "All average execution times saved as $OUTPUT."
```

Figure 7: process.sh

Running the process.sh and the output:



```
[akurt01@sariyer polybench-c-3.2 ]$ chmod +x process.sh
[akurt01@sariyer polybench-c-3.2 ]$ ./process.sh
All average execution times saved as output_times.txt.
[akurt01@sariyer polybench-c-3.2 ]$ cat output_times.txt
Benchmark: 2mm - Avg Time: 6.377 sec
Benchmark: 3mm - Avg Time: 9.576 sec
Benchmark: atax - Avg Time: 0.07 sec
Benchmark: bicg - Avg Time: 0.08 sec
Benchmark: doitgen - Avg Time: 0.39 sec
[akurt01@sariyer polybench-c-3.2 ]$
```

Figure 8: process.sh run

10. Results

The final execution times are stored in ‘output_times.txt‘:

```
Benchmark: 2mm - Avg Time: 5.469 sec
Benchmark: 3mm - Avg Time: 8.194 sec
Benchmark: atax - Avg Time: 0.082 sec
Benchmark: bicg - Avg Time: 0.102 sec
Benchmark: doitgen - Avg Time: 0.361 sec
```

11. Commands

Below is a summary of all the commands used throughout the process:

Step	**Command**
VPN Connection	sudo openvpn --config openvpn-uhem-config-linux.ovpn
SSH Connection to UHeM	ssh username@10.128.2.32
Download PolyBench-C	wget <URL>
Extract the Benchmark Suite	tar -xvzf polybench-c-3.2.tar.gz
Navigate to PolyBench Directory	cd polybench-c-3.2
Compile Benchmarks	gcc -O3 -I utilities -I -path- \ utilities/polybench.c c-path \ -DPOLYBENCH_TIME -o benchmark-name
Make compute.sh Executable	chmod +x compute.sh
Submit SLURM Job	sbatch slurm_sample.sh
Monitor SLURM Job	squeue -u $USER
Check SLURM Output	cat slurm-XXXXX.out
Check Benchmark Execution Output	cat output_shell.txt
Make process.sh Executable	chmod +x process.sh
Run process.sh	./process.sh
View Final Execution Times	cat output_times.txt

Table 1: Commands Used Throughout the Process

12. Conclusion

In this assignment, I successfully executed and analyzed PolyBench-C benchmarks on the UHEM system. Throughout the process, I learned how to compile, execute, and monitor benchmark programs.
The main steps I completed:

- Connecting to the UHEM system via VPN and SSH.
- Downloading and extracting the PolyBench-C benchmark.
- Compiling five selected benchmarks: `2mm`, `3mm`, `atax`, `bicg`, and `doitgen`.
- Writing and executing the `compute.sh` script, which ran each benchmark 10 times.
- Writing and executing the `process.sh` script to calculate the average execution time of each benchmark.